SQL Basics - Cheat Sheet

Contents

Reading rows

1. What SQL is
2. Basic SELECT
3. WHERE clause
4. Boolean logic
5. ORDER BY
6. LIMIT and OFFSET
7. DISTINCT

Changing rows

8. INSERT
9. UPDATE
10. DELETE

Joins and NULLs

11. Basic JOINs
12. NULL handling

Aggregation

13. Aggregate functions
14. GROUP BY
15. HAVING

Expressions and types

16. CASE expressions
17. Common data types
18. Conditional helpers

Reference

19. Defaults to follow
20. Common gotchas
21. Quick reference

1. What SQL is

SQL is a declarative, set-based language. We describe what data we want, and the database figures out how to fetch it. That's different from Go, where we tell the runtime each step.

"Set-based" matters. A query operates on rows as a group, not one at a time. If we find ourselves tracing a query row by row, we have the wrong model.

There's ANSI SQL (the standard) and then there are dialects:

Dialect	Notes
Postgres	Strictest of the five, and the dialect these examples target
MySQL	Lenient, and quotes identifiers with backticks rather than double quotes
SQL Server	Uses `TOP`, or ANSI `OFFSET` / `FETCH`, never `LIMIT`
Oracle	Uses `FETCH FIRST N ROWS ONLY`, and treats `''` as `NULL`
SQLite	Column types are hints, not constraints

Most basic `SELECT`, `INSERT`, `UPDATE` and `DELETE` statements are portable. Pagination, string functions, and date handling are where dialects diverge.

2. Basic SELECT

The shape of every query:

```
SELECT id, name, email
FROM users;
```

Column aliases with `AS`. The keyword is optional in all five dialects above, but spelling it out reads better:

```
SELECT name AS full_name, created_at AS signup_date
FROM users;
```

`SELECT *` grabs every column. That's fine for ad-hoc exploration. In production code it's risky, because adding a column upstream breaks consumers, so flag `SELECT *` in review.

3. WHERE clause

Filter rows with `WHERE`. The condition must evaluate to true for the row to be returned.

```sql
SELECT id, name
FROM users
WHERE status = 'active';
```

Comparison operators:

Operator	Meaning
`=`	Equal
`!=` or `<>`	Not equal (both are standard, `<>` is older ANSI)
`<`, `>`, `<=`, `>=`	Order numbers, and also dates and strings, by collation
`IN (...)`	Matches any value in a list
`BETWEEN x AND y`	Range, inclusive on both ends
`LIKE 'pattern'`	String pattern matching
`IS NULL` / `IS NOT NULL`	NULL check

Examples:

```sql
SELECT * FROM orders WHERE total > 100;
SELECT * FROM users WHERE status IN ('active', 'pending');
SELECT * FROM orders WHERE created_at BETWEEN '2025-01-01' AND '2025-01-31';
SELECT * FROM users WHERE email LIKE '%@gmail.com';
SELECT * FROM users WHERE deleted_at IS NULL;
```

`LIKE` wildcards: `%` matches any number of characters (including zero). `_` matches exactly one character. So `'A%'` finds anything starting with A. `'A_'` finds two-character strings starting with A.

4. Boolean logic

Combine conditions with `AND`, `OR`, `NOT`. `AND` binds tighter than `OR`, so `WHERE a AND b OR c` reads as `(a AND b) OR c`. Parenthesize whenever we mix them.

```sql
SELECT * FROM orders
WHERE status = 'pending'
  AND (total > 500 OR user_id IN (1, 2, 3));
```

5. ORDER BY

Sort the result. Default is `ASC`.

```sql
SELECT id, name, created_at
FROM users
ORDER BY created_at DESC;
```

Multiple columns:

```sql
SELECT * FROM orders
ORDER BY status ASC, created_at DESC;
```

NULL ordering varies by dialect. Postgres puts NULLs last by default for `ASC`. We can force it:

```sql
ORDER BY created_at DESC NULLS LAST;
```

MySQL doesn't support `NULLS FIRST/LAST` directly. We'd do `ORDER BY created_at IS NULL, created_at DESC` instead.

6. LIMIT and OFFSET

Paginate results. Postgres and MySQL:

```sql
SELECT * FROM users
ORDER BY id
LIMIT 20 OFFSET 40;
```

That gives us rows 41 to 60. Always pair `LIMIT` / `OFFSET` with `ORDER BY`. Without a sort clause the database can return rows in any sequence, and our pagination breaks.

Oracle and modern SQL Server use ANSI syntax:

```sql
SELECT * FROM users
ORDER BY id
OFFSET 40 ROWS
FETCH FIRST 20 ROWS ONLY;
```

A large `OFFSET` makes the database walk every skipped row before discarding it, so the cost grows with the offset, not with the page size. The intermediate sheet covers keyset pagination, which pays a flat cost per page.

7. DISTINCT

Strips duplicate rows from the result set.

```sql
SELECT DISTINCT status FROM orders;
```

`DISTINCT` applies to the whole row, not just one column. So `SELECT DISTINCT status, user_id` gives us unique combinations of both.

`DISTINCT` isn't free. The database has to sort or hash to deduplicate. If we're using it as a "fix" for a bad join, fix the join instead.

8. INSERT

Single row:

```sql
INSERT INTO users (name, email, status)
VALUES ('Alice', 'alice@example.com', 'active');
```

Multi-row:

```sql
INSERT INTO users (name, email, status)
VALUES
  ('Bob', 'bob@example.com', 'active'),
  ('Carol', 'carol@example.com', 'pending');
```

Insert from another query:

```sql
INSERT INTO archived_users (id, name, email)
SELECT id, name, email
FROM users
WHERE status = 'deleted';
```

If we skip the column list, we must supply values for every column in table order. That's brittle, name the columns.

9. UPDATE

Modify existing rows. Always include a `WHERE` clause unless we really mean to update every row.

```sql
UPDATE users
SET status = 'inactive'
WHERE last_login < '2024-01-01';
```

We can update multiple columns at once:

```
UPDATE orders
SET status = 'shipped', shipped_at = NOW()
WHERE id = 12345;
```

`UPDATE users SET status = 'inactive';` with no `WHERE` updates every row in the table. Run it inside `BEGIN; ... ROLLBACK;` first and check the row count before we commit.

10. DELETE

Same shape as UPDATE, same warning.

```
DELETE FROM users WHERE id = 42;
```

`DELETE FROM users;` (no `WHERE`) wipes every row. The table structure stays.

`TRUNCATE TABLE users;` empties a table fast, because it skips per-row logging. It can't be filtered with `WHERE`, and it won't run against a table that another table references by foreign key. Rollback splits by engine: Postgres can roll it back inside a transaction, MySQL and Oracle commit it implicitly. Use it for test-data resets.

11. Basic JOINs

Joins combine rows from two tables based on a matching column. Pretend we have:

```
users(id, name, email, created_at, status)
orders(id, user_id, total, status, created_at)
```

`INNER JOIN` returns rows where both sides match:

```
SELECT u.name, o.total
FROM users u
INNER JOIN orders o ON o.user_id = u.id;
```

If a user has no orders, they're excluded. If an order has no matching user, it's excluded.

`LEFT JOIN` keeps every row from the left table, even if the right side has no match. Missing columns come back as NULL.

```
SELECT u.name, o.total
FROM users u
LEFT JOIN orders o ON o.user_id = u.id;
```

That gives us every user, with NULL totals for users who never ordered. Useful for “find users without orders”:

```
SELECT u.id, u.name
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;
```

`RIGHT JOIN` is the mirror image. It’s rare in practice. Most people flip the table order and use `LEFT JOIN`.

`USING(col)` is shorter, but only when both tables name the join column identically. Our `users` and `orders` don’t qualify, because `users` has `id` where `orders` has `user_id`. Given a `payments` table that also keys on `order_id`:

```
SELECT o.total, p.method
FROM orders o
JOIN payments p USING (order_id);
```

`ON` is more flexible, because the column names can differ.

12. NULL handling

NULL is the absence of a value. It means “we don’t know”. It’s never equal to zero or an empty string, and any comparison with it returns unknown.

SQL uses three-valued logic: true, false, unknown.

```
SELECT NULL = NULL;         -- returns NULL (unknown), not true
SELECT NULL != 'something'; -- returns NULL
SELECT NULL = 0;            -- returns NULL
```

`NOT` doesn’t rescue it either. `NOT (NULL = 1)` is unknown, so the row still fails the filter.

So this query returns zero rows:

```
SELECT * FROM users WHERE deleted_at = NULL;          -- wrong
```

We need:

```
SELECT * FROM users WHERE deleted_at IS NULL;         -- right
```

NULLs in aggregates: `COUNT(*)` counts all rows. `COUNT(col)` ignores rows where col is NULL. `SUM`, `AVG`, `MIN`, `MAX` skip NULLs. So `AVG(price)` on five rows where one is NULL averages four values, not five.

13. Aggregate functions

Functions that collapse many rows into one value.

Function	Behavior
`COUNT(*)`	Count all rows
`COUNT(col)`	Count rows where col is not NULL
`COUNT(DISTINCT col)`	Count unique non-NULL values
`SUM(col)`	Sum, ignores NULLs
`AVG(col)`	Average, ignores NULLs
`MIN(col)`	Smallest value
`MAX(col)`	Largest value

```sql
SELECT
  COUNT(*)   AS total_orders,
  SUM(total) AS revenue,
  AVG(total) AS avg_order_value,
  MIN(total) AS smallest,
  MAX(total) AS largest
FROM orders
WHERE status = 'completed';
```

14. GROUP BY

Aggregate per group instead of across the whole table.

```sql
SELECT user_id, COUNT(*) AS order_count, SUM(total) AS spent
FROM orders
GROUP BY user_id;
```

> **Rule**
>
> Every column in the `SELECT` list that isn't inside an aggregate must appear in `GROUP BY`. Postgres and Oracle enforce this. So does MySQL, since 5.7, through `ONLY_FULL_GROUP_BY` in `sql_mode`. Switch that flag off and the server returns a value from an arbitrary row in the group.

```sql
-- Wrong: name is selected but not grouped or aggregated
SELECT user_id, name, COUNT(*)
FROM orders
GROUP BY user_id;
```

Grouping by `name` too, or wrapping it in `MIN(name)`, silences the error without fixing the model. Join to `users` and group by the key that owns the name:

```sql
SELECT u.id, u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;
```

15. HAVING

`WHERE` filters rows before grouping. `HAVING` filters groups after.

```sql
SELECT user_id, COUNT(*) AS order_count
FROM orders
WHERE status = 'completed'   -- filters individual orders
GROUP BY user_id
HAVING COUNT(*) > 5;         -- filters resulting groups
```

`WHERE` can't reference aggregates because the aggregation hasn't happened yet. That's why `HAVING` exists.

```sql
-- Wrong
SELECT user_id, COUNT(*) FROM orders
WHERE COUNT(*) > 5 GROUP BY user_id;

-- Right
SELECT user_id, COUNT(*) FROM orders
GROUP BY user_id HAVING COUNT(*) > 5;
```

16. CASE expressions

SQL's version of if/else. Two forms.

Simple form, like a switch:

```sql
SELECT id,
  CASE status
    WHEN 'active' THEN 'A'
    WHEN 'pending' THEN 'P'
    ELSE 'X'
  END AS status_code
FROM users;
```

Searched form, full conditions:

```sql
SELECT id, total,
  CASE
    WHEN total < 50 THEN 'small'
    WHEN total < 500 THEN 'medium'
    ELSE 'large'
  END AS bucket
FROM orders;
```

`CASE` works anywhere an expression works: `SELECT`, `WHERE`, `ORDER BY`, even `GROUP BY`.

```sql
SELECT
  CASE WHEN total < 100 THEN 'low' ELSE 'high' END AS tier,
  COUNT(*) AS cnt
FROM orders
GROUP BY CASE WHEN total < 100 THEN 'low' ELSE 'high' END;
```

17. Common data types

A quick map. Exact names vary by dialect.

Category	Common types
Integer	`INT`, `BIGINT`, `SMALLINT`
Decimal	`DECIMAL(p, s)` (exact), `NUMERIC` (alias)
Float	`REAL`, `DOUBLE PRECISION` (avoid for money)
String	`VARCHAR(n)`, `TEXT`, `CHAR(n)` (fixed length, rarely used)
Date/time	`DATE`, `TIME`, `TIMESTAMP`, `TIMESTAMPTZ` (with time zone)
Boolean	`BOOLEAN` (Postgres), `TINYINT(1)` (MySQL)
JSON	`JSON`, `JSONB` (Postgres, indexed binary form)

Don’t store money in floats. 0.1 + 0.2 isn’t 0.3 in IEEE 754. Use `DECIMAL(19, 4)` or similar.

For timestamps, prefer the time-zone-aware type when the dialect offers one. A zone that was never stored can't be recovered. Every later reader has to guess which zone the value was written in.

18. Conditional helpers

`COALESCE` returns the first non-NULL argument.

```
SELECT name, COALESCE(nickname, name) AS display_name
FROM users;
```

It earns its place after a `LEFT JOIN`, where the unmatched side comes back NULL: `COALESCE(SUM(o.total), 0)` reports zero for a user who never ordered, instead of nothing at all. The intermediate sheet covers `NULLIF`, `GREATEST`, `LEAST`, and string and date functions in more detail.

19. Defaults to follow

- Think in sets, not rows. A query that only makes sense as a loop is the wrong query.
- Always name the columns in INSERT, don't rely on table order.
- Always include `WHERE` in UPDATE and DELETE unless we really mean every row.
- Always pair `LIMIT` with `ORDER BY` for predictable pagination.
- Use `IS NULL` / `IS NOT NULL`, never `= NULL`.
- Parenthesize mixed `AND` / `OR` conditions.
- Avoid `SELECT *` in production queries. Name the columns we need.
- Use table aliases when joining, first letter of the table (`users u`, `orders o`).
- Format multi-line queries with one clause per line. We will read this code more often than we write it.

20. Common gotchas

- A `WHERE` clause on the right table silently turns a `LEFT JOIN` into an `INNER JOIN`, because NULL fails the predicate. Put the right-side filter in the `ON` clause if we want to keep the outer behavior. Testing for `IS NULL` is the exception, that's the anti-join idiom.
- `LIKE 'abc%'` can use an index. `LIKE '%abc'` (leading wildcard) usually can't.
- `OR` conditions across different columns often defeat index use. Watch query plans.
- `BETWEEN '2025-01-01' AND '2025-01-31'` misses anything on Jan 31 after midnight, because the upper bound is a timestamp at zero hours.

21. Quick reference

Task	Syntax
Select with filter	`SELECT col FROM tbl WHERE cond;`
Multiple conditions	`WHERE a = 1 AND (b = 2 OR c = 3)`
Pattern match	`WHERE name LIKE 'A%'`
NULL check	`WHERE col IS NULL`
Sort	`ORDER BY col DESC`
Paginate (Postgres/MySQL)	`LIMIT 20 OFFSET 40`
Paginate (Oracle/SQL Server)	`OFFSET 40 ROWS FETCH FIRST 20 ROWS ONLY`
Dedupe	`SELECT DISTINCT col FROM tbl`
Insert one	`INSERT INTO tbl (a, b) VALUES (1, 2);`
Insert many	`INSERT INTO tbl (a, b) VALUES (1,2), (3,4);`
Update	`UPDATE tbl SET a = 1 WHERE id = 5;`
Delete	`DELETE FROM tbl WHERE id = 5;`
Empty table fast	`TRUNCATE TABLE tbl;`
Inner join	`FROM a JOIN b ON a.id = b.a_id`
Outer join	`FROM a LEFT JOIN b ON a.id = b.a_id`
Count rows	`SELECT COUNT(*) FROM tbl;`
Group counts	`GROUP BY col`
Filter groups	`HAVING COUNT(*) > 5`
If/else	`CASE WHEN x THEN y ELSE z END`
Default for NULL	`COALESCE(a, b, 'default')`